Structured Query Language (SQL 2)

Primera Parte

Introducción

La primera publicación basada en la manipulación de los datos propuesta por Codd (creador del modelo relacional), fue desarrollada por un grupo de investigación de IBM, Chamberlain y Boyce en 1974, y dio origen a lo que se convirtió en SQL, llamado originalmente SEQUEL (***S***tructured ***E***nglish ***QUE***ry ***L***anguage). Dicho lenguaje era parte del SYSTEM R de IBM.

Proveedores de diferentes DBMS desarrollaron variantes de aquel lenguaje, hasta que finalmente ANSI e ISO generaron la versión estándar en 1986, llamada SQL-1 o SQL-86. Como se había omitido incorporar restricciones de integridad, tuvo que extenderse en el 89 y la mayoría de los proveedores desarrollaron DBMS compatibles con SQL-89. Dicha versión fue revisada y cambiada en 1992, generando SQL-92 o SQL-2. Actualmente está vigente SQL-99 o SQL-3, al que se le agregaron algunas extensiones en el 2003, 2006 y 2008 (Fulltext, GIS, integración con XML y XQuery, etc).

SQL es un híbrido entre álgebra relacional y cálculo relacional. Por un lado, desde la sintaxis del lenguaje, o sea desde el punto de vista del programador, tiene una gran influencia del Cálculo Relacional. Por otro lado, desde su implementación o sea desde el punto de vista de los motores DBMS, se utiliza el Algebra Relacional para reescribir la consulta SQL original y generar una equivalente pero más eficiente. Por tal motivo es un excelente ejemplo de cómo los fundamentos teóricos deben llevarse a la práctica.

Para los desarrolladores es muy importante trabajar con un DBMS SQLxx *compliant* porque garantiza que el trabajo puede ser migrado hacia otro DBMS que también sea SQLxx *compliant*, con escaso esfuerzo. Como se puede observar, el concepto de portabilidad también es necesario en lo que refiere a bases de datos.

La mayoría de los proveedores de servidores de base de datos (por ejemplo: DB2, Oracle, Progress, Sybase, Informix, PostgreSQL, etc.) han adoptado en bastante grado el estándar SQL-3 y han agregado sus propias extensiones. Si se quiere trabajar alternativamente con diferentes DBMS hay que tratar de usar exclusivamente los comandos ANSI-*compliant* y no sus características propietarias.

Como sabemos, un lenguaje de base de datos debe tener sentencias para la definición de los esquemas y también para la manipulación de su información. Comenzaremos analizando cada uno de sus grupos.

1. Data Definition Language (DDL)

Un ***esquema de base de datos*** contiene la definición de tablas, vistas, dominios y otras características (*assertions*, etc). El ***catálogo del sistema*** contiene el esquema de base de datos definido más cierta información propia (tablas y vistas relativas a dicha base de datos, cantidad de accesos a disco, uso de la memoria, etc).

El núcleo del modelo relacional lo constituyen las ***relaciones***, que están formadas por ***atributos***. Cada atributo toma sus valores de un ***dominio*** que restringen los valores que podrán tomar las tuplas en los mismos. Las ***restricciones*** del modelo relacional constituyen una forma fuerte de validación que libera a los programas que acceden a bases de datos de tener que ser ellos los que deban realizar los chequeos. Veremos cómo se representan estos conceptos en SQL.

Silvia Alicia Gómez & Leticia Irene Gómez

1.1. Tipos de Datos y su manipulación

Elegir el tipo de datos correcto para un atributo es muy importante porque no sólo establece la restricción de dominio del modelo relacional, sino que además **limita las operaciones** que pueden realizarse sobre los atributos e influye en el espacio que ocupará la tabla en disco. En otras palabras, garantiza la integridad semántica de la información y permite representar las **restricciones de dominio del modelo relacional**.

1.1.1 Tipos de Datos

SQL-2 ofrece un conjunto de dominios o tipos de datos ***built-in***. Según cuál sea el DBMS (DB2, Oracle, etc.) algunos tipos de datos podrían no estar disponibles o sus nombres podrían ser diferentes.

Tipo Carácter (Secuencia de caracteres, bytes todos del mismo conjunto CHARACTER SET).	
Nombre del tipo	***Descripción***
CHAR(tam) o **CHARACTER(tam)**	String de longitud fija. Si no se especifica tam se asume 1. Si se inserta una constante caracter de menor longitud, automáticamente se completa con blancos. El máximo tamaño posible depende del DBMS.
VARCHAR(max) o **CHAR VARYING(max)** o **CHARACTER VARYING(max)**	String de longitud variable. El tope está dado por el valor colocado entre paréntesis. El máximo tamaño posible depende del DBMS.

Tipo Numérico	
Nombre del tipo	***Descripción***
INTEGER o **INT**	Permite guardar enteros, cuyo rango está dado por la arquitectura de computadora. El máximo tamaño posible depende del DBMS.
SMALLINT	Permite guardar enteros más pequeños, cuyo rango también depende de la arquitectura de la computadora. El máximo tamaño posible depende del DBMS.
BIGINT	Permite guardar enteros, cuyo rango está dado por la arquitectura de computadora. En DB2 se usan 64 bits pasa su almacenamiento. En Oracle no está definido.
NUMBER(p,s) o **DECIMAL(p, s)** o **DEC(p,s)**	Número punto fijo donde p es la precisión y s la escala. La precisión p debe calcularse para que albergue la cantidad de dígitos de la mantisa + la cantidad de dígitos de la parte fraccionaria + un lugar para el signo. En realidad se guardan caracteres que representan número decimales. Si se quiere almacenar un número mayor no se puede, pero si quiere almacenar un número en el rango admitido pero con mayor escala, se redondea.
FLOAT(p)	Número punto flotante, con precisión indicada por el usuario, pudiéndose omitir. En DB2 es sinónimo de DOUBLE.
REAL	Número punto flotante, dependientes de la computadora.
DOUBLE	Número punto flotante con doble precisión, dependientes de la computadora

Silvia Alicia Gómez & Leticia Irene Gómez

Tipo Temporal	
Nombre del tipo	***Descripción***
DATE	Permite guardar una fecha con año, mes y día. Su implementación interna depende del DBMS.
TIME(fraction)	Permite guardar un tiempo con hora, minutos y segundos. En DB2 existe. En Oracle NO existe.
TIMESTAMP	Permite guardar fecha y tiempo con fracción de microsegundos (mínimo 5 lugares). Su implementación interna depende del DBMS.
INTERVAL	Representa la duración de un período. Permite sumar o restar el valor que almacena a los tipos de datos a los cuales refiere. DB2 no lo define como un nuevo tipo, ya que la resta de dos fechas directamente da un DECIMAL (Ejemplo: la resta de dos TIME es implícitamente un INTERVAL pero de tipo DECIMAL). O sea, no se puede definir una columna de este tipo, aunque la resta de dates da un número que representa los días transcurridos entre ellos. Ese valor numérico podría usarse para sumárselo a una fecha y obtener una desplazamiento. En Oracle existen 2 tipos de tipo intervalo llamados: INTERVAL YEAR TO MONTH, INTERVAL DAY TO SECOND. El primero expresa intervalos como años y meses. El segundo expresa intervalos como días, horas, minutos, segundos y fracciones de segundo.

1.1.2 Dominios

SQL-2 permite que el usuario ***cree un dominio*** con una sentencia DDL y luego declare las columnas asociándolas a dominios previamente creados. En realidad, el uso de DOMAIN sólo ofrece claridad, como el ***typedef*** del lenguaje C que otorga un alias a un dato ya existente.

Sintaxis

```
CREATE DOMAIN nombre_dominio [AS] built_in
    [DEFAULT valor_default]
    [restriccion];
```

donde restricción puede ser:

- NOT NULL
- CHECK(condición), típicamente condiciones establecidas sobre "value" con expresiones BETWEEN, LIKE, IN.

```
ALTER DOMAIN  nombre_dominio  dominio_alteration_action

DROP DOMAIN nombre_dominio
```

Silvia Alicia Gómez & Leticia Irene Gómez

Ejemplo:

CREATE DOMAIN nombreTipo CHAR(50) NOT NULL;
CREATE DOMAIN edadTipo INTEGER CHECK(value > 0);
CREATE DOMAIN colorTipo CHAR(4) CHECK(value IN ('rojo', 'azul'));
CREATE DOMAIN colorTipo2 CHAR(4) DEFAULT 'rojo' CHECK(value IN ('rojo', 'azul'));

Aclaración

PostgreSQL acepta la creación de dominios.

Ni DB2, ni Oracle soportan la definición a priori de dominios.

Dado que un dominio está caracterizado por:

- Tipo de datos (Ej: INTEGER, CHAR, etc)
- Rango (Ej: value entre 1000 y 2000)

cuando usamos algún DBMS que no permitan predefinir dominios, se lo puede materializar definiendo la restricción deseada **directamente en la definición de la columna de la tabla**. El único inconveniente es que si el mismo dominio fuera usado para distintas columnas de distintas tablas hay que recordar que si cambia alguna de ellas (ALTER) deben cambiarse también las otras definiciones.

1.1.3 Operadores y Funciones

Como dijimos previamente, los tipos de datos van a permitir cierto tipo de operadores, algunos de los cuales son polimórficos. Los parámetros de las funciones y su valor retornado también depende del dominio de los datos. Citamos sólo los más usados:

- **Operador** para concatenar strings (chars y varchars): ||

- **Operadores** para datos aritméticos: +, -, * y /

  Ejemplo:

```
SELECT ISBN, precio
FROM libro
WHERE precio * 0.2 > 100
```

- **Operadores** para datos temporales: +, -

  La resta de 2 tipos temporales (date, time, timestamp) devuelve un intervalo. Ese valor puede sumarse a otra de tipo temporal (date, time, timestamp) para desplazarlo.

- **Funciones** para tipos temporales TIME, DATE, TIMESTAMP, INTERVAL: Encontramos que no sólo los DBMS implementan diferente los tipos, sino que los operadores y funciones difieren entre sí. Esto complica la portabilidad.

Silvia Alicia Gómez & Leticia Irene Gómez

- SQL define las siguientes funciones escalares no determinísticas y sin argumentos para recuperar la fecha, la hora o ambos (retornan un valor de dominio date, time y timestamp, respectivamente): **current_date, current_time, current_timestamp**.

  Ejemplo:

```
SELECT current_date, ISBN
FROM libro
```

- SQL define las siguiente función para extraer una parte de cierta expresión temporal: EXTRACT(parte FROM temporalexpresion) . Esta función siempre devuelve valores numéricos.

  Ejemplo: Si la fecha actual fuera Abril del 2013, devolvería 4 tantas veces como tuplas posee la tabla libro

```
SELECT EXTRACT( month  FROM  current_date)
FROM libro
```

1.2. Tablas Persistentes definidas a partir de columnas

Precisamos conocer cómo definir las relaciones del modelo relacional y sus **restricciones de clave, integridad de entidad** e **integridad referencial**.

Las tablas pueden ser persistentes o temporarias. En este ítem analizaremos sólo las persistentes.

Muy Importante

SQL, a diferencia de un lenguaje verdaderamente relacional, permite que las tablas no tengan claves definidas, y por lo tanto permiten que las tuplas estén repetidas. Se dice que maneja "***multisets*** *o* ***bags***", en vez de "***sets***".

Sintaxis

```
CREATE TABLE nombre_tabla
(
  nombre_atributo1 tipo1     [DEFAULT default1]     [restriccion1],
  nombre_atributo2 tipo2     [DEFAULT default2]     [restriccion2],
     ...
  nombre_atributoN tipoN     [DEFAULT defaultN]    [restriccionN],

  [ [CONSTRAINT nombre_restriccion1] PRIMARY KEY( lista_de_atributos) ],
  [ [CONSTRAINT nombre_restriccion2] UNIQUE( lista_de_atributos) ],
  [ [CONSTRAINT nombre_restriccion3] FOREIGN KEY (lista_atributosLocal)
                 REFERENCES tabla_referenciada  [ (lista_atributosRef) ]
          [ON DELETE expresion1]  [ON UPDATE expresion2]  ]
);
```

Silvia Alicia Gómez & Leticia Irene Gómez

donde en cada atributo:

- ***tipo*** indica un tipo ***built-in*** o algún dominio previamente definido
- ***default*** permite indicar un valor por defecto. Si esto no se indica explícitamente se asume que el valor default es automáticamente NULL.
- ***restriccion*** permite indicar (restricciones más avanzadas y complejas que estas pueden expresarse en SQL3 como veremos en apuntes posteriores):
  - NOT NULL
  - [CONSTRAINT nombre_opcional] **PRIMARY KEY o UNIQUE**, donde el nombre opcional se indica si luego se quiere borrar dicha restricción.

y donde las restricciones globales son:

- **CONSTRAINT *nombre_restriccion***: Opcionalmente se puede dar un nombre especial a cada una de las restricciones. El nombre se usa cuando se desea borrar dicha restricción.

- **PRIMARY KEY (*lista_atributos*)**: Permite citar la lista de atributos que forman una clave identificatoria de esta tabla. Sólo una cláusula de este tipo puede aparecer. Como la clave primaria también puede indicarse directamente agregando la palabra PRIMARY KEY al lado del atributo, **sólo una de estas alternativas puede ser usada**. Cuando la clave primaria está formada por un solo atributo se puede usar cualquiera de las dos modalidades. Cuando está formada por un conjunto de atributos, sólo se debe hacer especificando la restricción en forma global (no al lado del atributo). Los atributos que conforman la clave primaria deben ser NO NULL. En PostgreSQL se puede omitir (lo traduce automáticamente a NOT NULL).

- **UNIQUE(*lista_atributos*)**: Indica que esa lista de atributos no admite repeticiones. Con esta restricción **podemos (y debemos)** representar TODAS claves candidatas (no sólo una de ellas como clave primaria). **Es muy importante HACERLO!**

MUY Importante

El modelo relacional identifica una o varias claves para cada relación. TODAS las claves deben expresarse en las tablas.

Es anecdótico (histórico desde el primer SQL) que exista una sola PRIMARY KEY pero tantas UNIQUE como queramos. Es más, es por compatibilidad que se mantuvo una palabra aparte que fuera PRIMARY KEY.

Siempre, Siempre, Siempre:

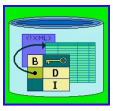
"Representar TODAS las claves de la siguiente forma:

- **Una cualquiera como Primary Key**
- **Todas las demás como UNIQUE declarando como restricción en cada uno de los atributos que la integran que no puede ser NULL. No olvidar esto."**

De esta forma podemos asegurar que:
"PRIMARY KEY sobre una lista de atributos es lógicamente equivalente a UNIQUE con NOT NULL en cada uno de esos atributos".

Si omitimos el NOT NULL en los atributos UNIQUE no resulta equivalente a un PRIMARY KEY porque los "nulls" no pueden formar parte de una CLAVE según el modelo relacional: "***Restricción de Integridad de Entidad***"

Silvia Alicia Gómez & Leticia Irene Gómez



- **FOREIGN KEY(*lista_atributosLocal)* REFERENCES *tabla_referenciada* (*lista_atributosRef*)**: Permite especificar una restricción de integridad referencial. Pueden usarse varias de estas cláusulas. La primera lista de atributos pertenece a la tabla que está definiendo dicha restricción, y la segunda lista nombra una clave candidata (ya sea declarada como primary key o unique) de la tabla a la cual ella referencia. Por default, se referencia la clave primaria de cierta tabla y como la misma es única no hace falta colocar explícitamente la lista de sus atributos. Si en cambio si se referencia una clave unique, debe colocarse explícitamente la lista de atributos, ya que puede haber varias de estas claves.

  SQL-2 agregó la posibilidad de especificar acciones a tomar en el caso de una eliminación o actualización de la tupla externa referenciada. Dicha acción se expresa por una expresión que puede contener: SET NULL, SET DEFAULT, CASCADE, NO ACTION (o RESTRICT). En el caso de que se borre una tupla y esta sea referenciada desde otra tabla, las acciones indican qué acción seguir en dicha referencia. La primera permite "romper" la referencia ya que deja que los atributos no tomen el valor de alguna tupla en la tabla referenciada. La segunda permite setear los valores a un cierto default. La tercera, en cambio, constituye la idea original de una clave foránea: si es un borrado se deben eliminar en cascada las tuplas que referencian las filas desaparecidas en la relación referenciada, o si es una actualización se deben cambiar sus valores para que coincidan con los de las tuplas referenciadas que cambiaron. La última (default) rechaza la operación.

Obviamente también se tienen sentencias para destruir lo previamente creado.

Sintaxis

```
DROP TABLE nombre_tabla [CASCADE | RESTRICT];
```

CASCADE implica que se borra también cualquier otra tabla o vista que la esté referenciando.
RESTRICT sólo permite borrar la tabla si ninguna otra tabla o vista la referencia.

Además tenemos sentencias DDL para *cambiar la estructura* ya definida. Los cambios que se permiten realizar sobre una tabla ya definida son: agregar atributos, eliminar atributos, cambiar la definición de un atributo, agregar restricciones, eliminar restricciones.

Sintaxis

```
ALTER TABLE nombre_tabla ADD nombre_atributo tipo
                                        [default] [restriccion];
```

o bien

```
ALTER TABLE nombre_tabla DROP nombre_atributo [CASCADE | RESTRICT];
```

o bien

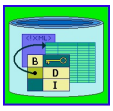
```
ALTER TABLE nombre_tabla ALTER nombre_atributo DROP DEFAULT;
```

o bien

```
ALTER TABLE nombre_tabla ALTER nombre_atributo SET DEFAULT valor;
```

o bien

Silvia Alicia Gómez & Leticia Irene Gómez



```
ALTER TABLE nombre_tabla DROP CONSTRAINT nombre_restriccion
                                          [CASCADE | RESTRICT];
```

o bien

```
ALTER TABLE nombre_tabla ADD CONSTRAINT nombre_restriccion  defincion;
```

En cuanto a la eliminación de atributos, SQL-2 permite especificar CASCADE y RESTRICT. La primera se usa para borrar todas las restricciones y vistas que referencian a dicho atributo. La segunda produce la destrucción de la columna si no hay restricción ni vista que referencie a dicha columna.

En el caso de agregar una restricción puede tener el formato citado previamente en la sintaxis del CREATE TABLE.

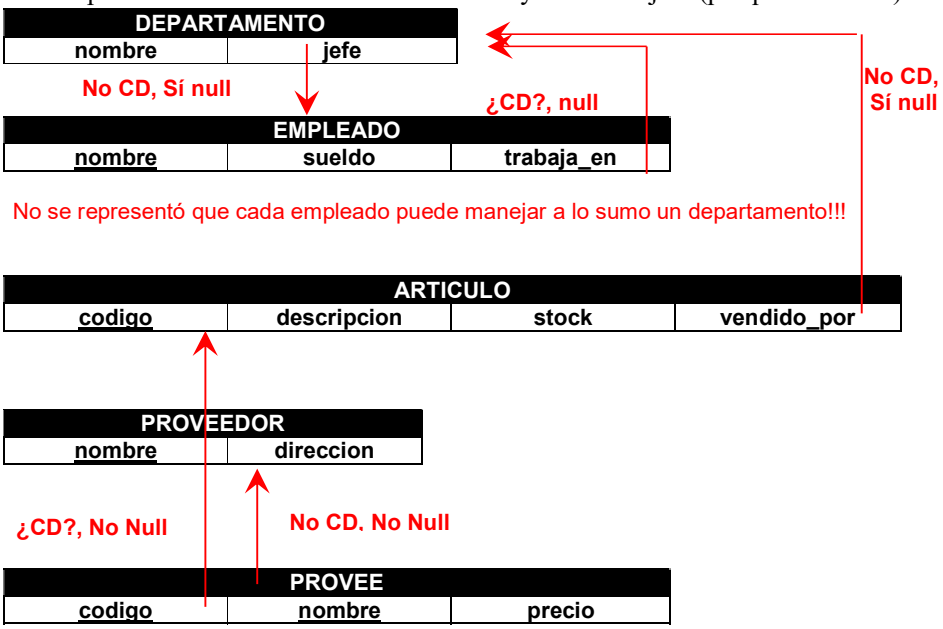
Ejemplo:

Volvamos al *ejercicio 1* del *TP 2*, donde se debía armar el esquema para el diagrama de entidad/relación del *TP 1*. Antes de comenzar la implementación (definición del esquema) debemos sacarnos todas las dudas que pudieron quedar pendientes. Supongamos que realizamos las últimas consultas:

- ¿Si el departamento desaparece, el empleado desaparece? Supongamos que NO. Entonces no debe haber borrado en cascada.
- ¿Si el artículo desaparece, interesa seguir teniendo la asociación de lo provisto? Supongamos que NO. Entonces debe haber borrado en cascada.
- Los proveedores nunca desaparecen porque sus datos siempre interesan, según decía el enunciado

Silvia Alicia Gómez & Leticia Irene Gómez

Departamento tiene 2 claves: una nombre y la otra en jefe (porque era único)



Podríamos pensar que la traducción de las primeras dos tablas y sus restricciones podrían formularse así:

```
CREATE TABLE departamento
(
   nombre    CHAR(50) NOT NULL ,
   jefe      CHAR(100),
   UNIQUE (jefe),
   PRIMARY KEY(nombre),
   FOREIGN KEY(jefe) REFERENCES empleado
);

CREATE TABLE empleado
(
   nombre    CHAR(100)  NOT NULL,
   sueldo    DECIMAL(10,2),
   trabaja_en  CHAR(50),
   PRIMARY KEY (nombre),
   FOREIGN KEY(trabaja_en) REFERENCES departamento
);
```

Con esto representamos la restricción de que un empleado no puede ser jefe de más de un departamento

Mal: no puede referenciarse a la tabla empleado porque todavía no fue definida. Tampoco podemos cambiar de orden la definición de estas tablas porque la tabla empleado también referencia a la tabla departamento

Este problema corresponde a la "especificación de claves foráneas cuando dos tablas se referencian entre sí".

Una solución consiste en crear una de las tablas primero, sin su clave foránea, luego crear la segunda tabla con todas las restricciones y finalmente modificar la primera tabla (alter table) para agregar la clave foránea faltante. Esto soluciona el problema de la "creación de las claves foráneas" pero no necesariamente el de la "inserción de tuplas en ambas tablas". ¿Qué ocurrirá si ninguna de las dos tablas acepta que el campo sea NULL? No se podrá insertar en una tabla porque deberá insertarse primero en la otra tabla, y viceversa.

Silvia Alicia Gómez & Leticia Irene Gómez

Este problema se soluciona armando una transacción que involucre la inserción de las dos tablas pero solicitando, al iniciar la transacción, que "difiera" la validación hasta que la transacción finalice. Para ese entonces ya estarán insertadas ambas tuplas y cada una referirá a la otra sin problemas. **Vamos a discutir la sintaxis de este formato cuando estudiemos transacciones.**

Como en este ejercicio, tanto el atributo ***jefe*** de la tabla ***departamento*** como el atributo ***trabaja_en*** en la tabla ***empleado*** pueden ser null, podemos seguir con este esquema sin problema (no necesitamos usar una transacción para la inserción con referencia cruzada):

```
CREATE TABLE departamento
(
  nombre      CHAR(50)  NOT NULL,
  jefe        CHAR(100) NOT NULL,
  UNIQUE(jefe),
  PRIMARY KEY(nombre)
 );

CREATE TABLE empleado
(
  nombre      CHAR(100)   NOT NULL,
  sueldo      DECIMAL(10,2),
  trabaja_en  CHAR(50),
  PRIMARY KEY(nombre),
  FOREIGN KEY(trabaja_en) REFERENCES departamento
 );
ALTER TABLE departamento ADD CONSTRAINT deptFK FOREIGN KEY(jefe)
            REFERENCES empleado;

CREATE TABLE articulo
(
  codigo      INTEGER   NOT NULL,
  descripcion CHAR(100) NOT NULL,
  stock         INTEGER   NOT NULL,
  vendido_por CHAR(50),
  PRIMARY KEY(codigo),
  FOREIGN KEY(vendido_por) REFERENCES departamento
);

CREATE TABLE proveedor
(
  nombre      CHAR(100) NOT NULL,
  direccion   CHAR(50)  NOT NULL,
  PRIMARY KEY(nombre)
);


CREATE TABLE provee
(
  codigo INTEGER   NOT NULL,
  nombre CHAR(100) NOT NULL,
  precio FLOAT,
  PRIMARY KEY(codigo,nombre),
  FOREIGN KEY(codigo) REFERENCES articulo ON DELETE CASCADE,
  FOREIGN KEY(nombre) REFERENCES proveedor);
```

Silvia Alicia Gómez & Leticia Irene Gómez

2. *Data Manipulation Language (DML)*

Tenemos sentencias para poder modificar las relaciones: insertar, eliminar, actualizar tuplas, así como también un potente conjunto de sentencias para expresar consultas.

2.1. *Inserción de Tuplas*

Comenzaremos con el ingreso de información a las relaciones ya creadas:

Sintaxis

```
INSERT INTO nombre_tabla [ (lista_atributos) ] values (lista_valores);
```

Este formato permite insertar **una tupla por vez**. La lista de atributos puede omitirse siempre y cuando se inserten valores para cada uno de los atributos definidos en el esquema y en el orden en que figuran en el esquema.

Si cuando se inserta una tupla no se especifican ciertos valores y se ***definió algún valor default*** para ese atributo, entonces el DBMS automáticamente setea dicho valor en su default. Si en cambio no se hubieran definido valores default, el DBMS coloca sus valores en NULL, salvo que se hubiera indicado que dicha columna no podía ser NULL, en cuyo caso el DBMS devuelve un error y no se produce la inserción.

Tener en cuenta que en el momento de una inserción el DBMS chequea las restricciones de dominio, claves, claves foráneas, o cualquier otra restricción que se hubiera indicado.

***Ejemplo*:** Sea el siguiente esquema

```
CREATE TABLE alumno
(
     codigo    INTEGER NOT NULL,
     nombre    CHAR(100) DEFAULT 'Ana'
     telefono  CHAR(50)
);
```

Al ejecutar la sentencia SQL

```
INSERT INTO  alumno (codigo) VALUES (10);
```

se obtendría

ALUMNO		
codigo	nombre	telefono
10	Ana	null

En cambio, al ejecutar la sentencia SQL

```
INSERT INTO  alumno (telefono, nombre)  VALUES ('4314-7778', 'ITBA');
```

se obtiene un error porque falta indicar el valor del atributo codigo que fue declarado ***not null.***

Silvia Alicia Gómez & Leticia Irene Gómez

2.2. Borrado de Tuplas

La eliminación de tuplas previamente creadas

Sintaxis

```
DELETE FROM nombre_tabla [ WHERE condicion ];
```

Se pueden borrar todas las tuplas de una relación, o sólo las tuplas que verifiquen la condición citada.

Obviamente el borrado de tuplas puede ocasionar cierta acción por parte del DBMS, como por ejemplo si se ha indicado borrado en cascada para una clave foránea que referencia la tabla de la cual se desean borrar tuplas. También puede resultar que el DBMS devuelva un error, por ejemplo cuando no se definió borrado en cascada y hay una tabla con clave foránea que posee valores coincidentes con la tupla a borrar.

Ejemplo:

Si en la relación anterior se tuvieran los siguientes datos

ALUMNO		
codigo	nombre	telefono
10	Ana	null
20	Juan	314-7778
0	Jose	313-8712
40	Lita	null
50	Lila	null
60	Alex	314-7778

y se ejecutara

```
DELETE FROM alumno WHERE codigo < 20;
```

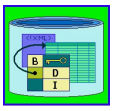
la relación se modificaría

ALUMNO		
codigo	nombre	telefono
20	Juan	314-7778
30	Jose	313-8712
40	Lita	null
50	Lila	null
60	Alex	314-7778

Si ahora se solicitara

```
DELETE FROM alumno WHERE telefono IS NULL;
```

Silvia Alicia Gómez & Leticia Irene Gómez



la relación quedaría

ALUMNO		
codigo	nombre	telefono
20	Juan	314-7778
30	Jose	313-8712
60	Alex	314-7778

Si finalmente se ejecutara la siguiente sentencia, la relación quedaría vacía (0 tuplas).

```
DELETE FROM alumno;
```

2.3. Actualización de Tuplas

Consiste en cambiar el valor de algunos de los atributos de tuplas previamente creadas.

Sintaxis

```
UPDATE nombre_tabla SET lista_cambios [WHERE condicion ];
```

donde lista_cambios es una lista formada por pares de asignación: atributo= nuevo_valor

Se pueden actualizar todas las tuplas de una relación, o sólo las tuplas que verifiquen la condición citada.

Obviamente la actualización de tuplas puede ocasionar cierta acción por parte del DBMS, como por ejemplo si se ha indicado set null para una clave foránea que referencia la tabla de la cual se desean actualizar tuplas. También puede resultar que el DBMS devuelva un error.

Ejemplo

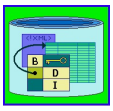
Supongamos que la relación anterior tiene los siguientes datos

ALUMNO		
codigo	nombre	telefono
10	Ana	null
20	Juan	314-7778
30	Jose	313-8712
40	Lita	null
50	Lila	null
60	Alex	314-7778

Al ejecutar la sentencia SQL

```
UPDATE ALUMNO  SET telefono = '4321-1211' WHERE telefono IS NULL;
```

Silvia Alicia Gómez & Leticia Irene Gómez



se obtendría

ALUMNO		
codigo	nombre	telefono
10	Ana	4321-1211
20	Juan	314-7778
30	Jose	313-8712
40	Lita	4321-1211
50	Lila	4321-1211
60	Alex	314-7778

Al ejecutar

```
UPDATE alumno SET nombre = 'Quien';
```

se obtendría

ALUMNO		
codigo	nombre	telefono
10	Quien	4321-1211
20	Quien	314-7778
30	Quien	313-8712
40	Quien	4321-1211
50	Quien	4321-1211
60	Quien	314-7778

Finalmente, al ejecutar

```
UPDATE alumno SET nombre = 'Otro', telefono=null;
```

se obtendría

ALUMNO		
codigo	nombre	telefono
10	Otro	null
20	Otro	null
30	Otro	null
40	Otro	null
50	Otro	null
60	Otro	null

Muy Importante

Notar que NO se puede preguntar en las condiciones si "atributo = NULL" sino que hay que consultar por "**atributo IS NULL**" (ídem con distinto), ya que nunca un NULL se considera igual a otro.

Sin embargo, cuando se quiere setear una columna con el valor NULL se lo asigna directamente, tanto en el *insert* como en el *update* (porque no es una condición booleana).

Silvia Alicia Gómez & Leticia Irene Gómez